분기한정 기법

현재까지의 비용만 봐도 더 좋은 답이 될 수 없으면 그 갈래를 멈춥니다.

분기한정

분기한정

현재까지의 비용만 봐도 더 좋은 답이 될 수 없으면 그 갈래를 멈춥니다.

예: 이미 최저 비용 7을 찾음

다른 갈래의 현재 비용이 8이고 앞으로 비용을 더해야 한다면 멈춥니다.

8-1 주어진 방문 순서의 왕복 비용

도시 사이의 이동 비용 표 dist와 방문 순서 route가 주어진다. route의 첫 도시는 0이며 모든 도시가 한 번씩 들어 있다. 이 순서로 이동한 뒤 마지막 도시에서 0번 도시로 돌아온다.

경로 전체의 이동 비용을 더해 반환하시오.

8-1 입력과 반환값

Python: route_cost(dist, route)

dist: 도시 1~9개의 정사각 비용 표. dist[i][j]는 i에서 j로 가는 비용
route: 0에서 시작하고 0~n-1을 한 번씩 포함하는 길이 n의 배열

반환값

주어진 경로의 왕복 비용을 정수 하나로 반환한다.

선택·판정 조건

모든 도시 사이를 이동할 수 있다. 비용은 0~100이고 대각선은 0이다.
i에서 j로 가는 비용과 j에서 i로 가는 비용은 달라도 된다.
방문 순서는 바꾸지 않는다. 마지막 귀환 비용도 합에 포함한다.

8-1 0,1,2 순서 · 방문 순서를 반대로

0,1,2 순서

dist=[[0,2,8],[1,0,3],[4,6,0]], route=[0,1,2]
반환값: 9
0→1의 2, 1→2의 3, 2→0의 4를 더한다.

방문 순서를 반대로

같은 dist, route=[0,2,1]
반환값: 15
이번에는 8+6+1=15이다. 앞 경로와 같은 값일 필요가 없다.

8-1 두 도시 왕복 · 도시 하나

두 도시 왕복

dist=[[0,5],[8,0]], route=[0,1]
반환값: 13
갈 때 5, 돌아올 때 8을 더한다.

도시 하나

dist=[[0]], route=[0]
반환값: 0
자기 자신으로 돌아가는 비용은 0이다.

행은 출발 도시, 열은 도착 도시

출발	도착0	도착1	도착2
도시0	0	2	8
도시1	1	0	3
도시2	4	6	0

0→1은 2, 1→0은 1이다. 반대 방향의 비용을 같다고 가정하지 않는다.

8-1 예제의 계산 과정

이동	비용 표의 칸	비용	누적 합
0→1	dist[0][1]	2	2
1→2	dist[1][2]	3	5
2→0	dist[2][0]	4	9

route=[0,1,2]의 비용은 2+3+4=9이다.

8-1 비용 표의 행은 출발, 열은 도착

dist[1][2]는 1번 도시에서 2번 도시로 가는 비용이다. route=[0,1,2]라면 dist[0][1], dist[1][2], dist[2][0]을 읽는다. 마지막 배열 위치 다음에는 route[0]을 읽어 출발 도시로 돌아오게 한다.

8-1 코드에서 사용할 값

코드	route=[0,1,2]일 때
i=0	dist[0][1]을 더함
i=1	dist[1][2]를 더함
i=2	dist[2][0]을 더함
(i+1)%3	다음 번호는 1,2,0. 마지막에 처음으로 돌아옴

행은 출발 도시, 열은 도착 도시입니다.

8-1 Python

```
def route_cost(dist, route):
    total = 0
    for i in range(len(route)):
        total += dist[route[i]][route[(i+1)%len(route)]]
    return total
```

시간 O(n), 추가 공간 O(1)

8-1 C 함수의 인자

함수 원형

int route_cost(const int dist[][9], int n, const int route[])

배열과 작업 공간

dist는 비용 표, n은 도시 수, route는 방문 순서이다. route의 길이는 n이다.

8-1 C

```
int route_cost(const int dist[][9], int n, const int route[]) {
    int total=0;
    for (int i=0; i<n; ++i)
        total+=dist[route[i]][route[(i+1)%n]];
    return total;
}
```

시간 O(n), 추가 공간 O(1)

8-2 남은 작업 비용의 하한 계산

사람 n명과 작업 n개의 배정 비용 표 cost가 있다. 행은 사람, 열은 작업이다. 0번부터 next_row−1번 사람까지는 이미 작업을 맡겼다. 아직 배정하지 않은 각 사람에 대해, 사용하지 않은 작업 중 가장 싼 비용을 하나씩 골라 더한다.

이 규칙으로 계산한 남은 비용의 합을 반환하시오.

8-2 입력과 반환값

Python: assignment_bound(cost, used, next_row)

cost: 1~8 크기의 정사각 비용 표. 각 비용은 0~100
used: 작업을 이미 맡겼으면 1, 아니면 0인 길이 n의 배열
next_row: 다음에 배정할 사람 번호. 0~n이며 used의 1 개수와 같음

반환값

남은 사람들의 행별 최소 비용을 더한 정수를 반환한다.

선택·판정 조건

이미 사용한 작업 열은 제외한다.
이 계산에서는 서로 다른 행에서 같은 작업 열을 골라도 허용한다. used를 추가로 바꾸지 않는다.
중복을 허용해 낮게 잡은 금액이므로 실제 가능한 최소 비용보다 커지지 않는다. 이런 값을 하한이라고 한다.
next_row=n이면 배정이 끝난 상태이므로 0을 반환한다.

8-2 일부 작업 사용 · 가장 싼 열이 겹침

일부 작업 사용

cost=[[9,2,7],[6,4,3],[5,8,1]], used=[0,1,0], next_row=1
반환값: 4
작업1을 빼고 남은 행의 최소 3과 1을 더한다. 둘 다 작업2의 값이다.

가장 싼 열이 겹침

cost=[[1,8],[2,9]], used=[0,0], next_row=0
반환값: 3
각 행에서 작업0을 골라 1+2=3이다. 실제 배정에서는 이 중복을 허용하지 않는다.

8-2 사용한 열 제외 · 배정 완료

사용한 열 제외

같은 cost, used=[1,0], next_row=1
반환값: 9
남은 사람1은 작업0을 고를 수 없어 작업1의 비용 9를 쓴다.

배정 완료

cost=[[7]], used=[1], next_row=1
반환값: 0
남은 사람이 없으므로 더할 비용도 없다.

8-2 예제의 계산 과정

사람	작업 0	작업 1	작업 2	고른 최소
0	9	2: 이미 배정	7	이번 계산에서 제외
1	6	4: 사용 불가	3	3
2	5	8: 사용 불가	1	1

서로 다른 사람에게 작업 2를 중복해서 골랐습니다. 비용을 낮게 잡기 위한 계산입니다.

8-2 하한 3과 실제 최소 비용 10의 차이

[[1,8],[2,9]]의 행별 최소는 1과 2여서 하한은 3이다. 실제 배정은 작업을 한 번씩만 써야 하므로 1+9 또는 8+2이고 둘 다 10이다. 중복을 잠시 허용한 계산은 가능성을 낮게 잡는 데 쓰고, 완성된 배정의 비용은 별도로 계산한다.

왜 4가 완성 배정의 답은 아닐까요?

같은 작업을 두 사람에게 맡길 수 없기 때문입니다.

가능한 실제 배정	비용 합
1번 사람→작업0, 2번 사람→작업2	6+1=7
1번 사람→작업2, 2번 사람→작업0	3+5=8
실제 남은 최소 비용	7

낮게 잡은 계산은 4, 실제 가능한 최소 비용은 7입니다.

8-2 코드에서 사용할 값

변수	예제에서의 뜻
next_row=1	1번 사람부터 남아 있음
used=[0,1,0]	작업 1은 이미 사용, 0과 2는 사용 가능
low	이번 사람에게 가능한 가장 싼 비용
answer	사람별 low를 더한 값

이 값처럼 실제 최소 비용보다 크지 않게 잡은 값을 하한이라고 부릅니다.

8-2 Python

```
def assignment_bound(cost, used, next_row):
    n = len(cost)
    answer = 0
    for r in range(next_row, n):
        low = 10**9
        for c in range(n):
            if used[c] == 0:
                low = min(low, cost[r][c])
        answer += low
    return answer
```

시간 $O(n^2)$, 추가 공간 $O(1)$

8-2 C 함수의 인자

함수 원형

int assignment_bound(const int cost[][8], int n, const int used[], int next_row)

배열과 작업 공간

cost는 비용 표, n은 사람 수이자 작업 수이다. used와 next_row는 Python과 같은 뜻이다.

8-2 C

```
int assignment_bound(const int cost[][8], int n,
    const int used[], int next_row) {
    int answer=0;
    for (int r=next_row; r<n; ++r) {
        int low=1000000;
        for (int c=0; c<n; ++c)
            if (!used[c] && cost[r][c]<low) low=cost[r][c];
        answer+=low;
    }
    return answer;
}
```

시간 $O(n^2)$, 추가 공간 $O(1)$

8-3 작업을 중복 없이 배정하는 최소 비용

사람 n명에게 서로 다른 작업 n개를 하나씩 맡긴다. 사람마다 정확히 한 작업을 맡고, 각 작업도 정확히 한 사람에게 배정해야 한다. cost[r][c]는 사람 r이 작업 c를 맡을 때의 비용이다.

가능한 모든 배정 중 비용 합의 최솟값을 반환하시오.

8-3 입력과 반환값

Python: assignment_min(cost)

cost: 1~8 크기의 정사각 비용 표. 각 비용은 0~100인 정수

반환값

모든 사람에게 작업을 배정한 총비용의 최솟값을 반환한다. 배정 목록은 반환하지 않는다.

선택·판정 조건

같은 작업을 두 사람에게 맡길 수 없다.
행마다 가장 싼 값을 따로 골라 더하면 중복 작업이 생길 수 있다. 실제 배정의 비용은 이 조건까지 지켜 계산한다.
비용 0도 정상적인 배정 비용이다.

8-3 두 사람 배정 · 최소 비용의 열이 겹침

두 사람 배정

cost=[[4,1],[2,3]]
반환값: 3
사람0에게 작업1, 사람1에게 작업0을 맡기면 1+2=3이다.

최소 비용의 열이 겹침

cost=[[1,8],[2,9]]
반환값: 10
1과 2는 같은 작업0의 비용이다. 가능한 배정은 1+9 또는 8+2로 둘 다 10이다.

8-3 비용이 모두 0 · 사람 한 명

비용이 모두 0

cost=[[0,0],[0,0]]
반환값: 0
어느 완성 배정이나 비용은 0이다.

사람 한 명

cost=[[7]]
반환값: 7
유일한 작업을 그 사람에게 맡긴다.

사람과 작업의 비용 표

행은 사람, 열은 작업입니다.

사람	작업0	작업1
사람0	4	1
사람1	2	3

사람0에게 작업1을 맡기면 1, 사람1에게 작업0을 맡기면 2입니다.

8-3 예제의 계산 과정

사람0의 작업	사람1의 작업	비용 합
0	1	4+3=7
1	0	1+2=3
최솟값		3

사람0과 사람1이 모두 작업1을 고르는 배정은 허용하지 않습니다.

8-3 이미 찾은 비용보다 비싼 갈래는 중단한다

비용 7인 완성 배정을 이미 찾았다고 하자. 다른 배정의 현재 비용이 8이면 앞으로 작업을 더 맡겨도 7보다 작아질 수 없다. 모든 비용이 0 이상이기 때문이다. 이때 남은 작업을 모두 시도하지 않고 돌아간다. 다른 배정을 볼 때는 방금 쓴 작업의 사용 표시를 지운다.

현재 비용이 8이면 더 볼 필요가 없습니다

이미 비용 7짜리 완성 배정을 찾았다고 합시다.

남은 비용이 0이어도

현재 8+남은 0=8입니다. 이미 찾은 7보다 비쌉니다.

이 갈래의 처리

남은 작업을 전부 배정하기 전에 탐색을 중단합니다.

다음 갈래

방금 고른 작업의 사용 표시를 지우고 다른 작업을 시도합니다.

8-3 코드에서 사용할 값

변수	뜻
row	이번에 작업을 맡길 사람 번호
used[c]	작업 c를 이미 맡겼으면 1
total	지금까지 배정한 비용의 합
best	이미 완성한 배정 중 가장 작은 비용

비용이 전부 0 이상이므로, 앞으로 작업을 더 맡겨도 total은 줄어들지 않습니다.

8-3 Python (1/2)

```
def assignment_min(cost):
    n = len(cost)
    used = [False]*n
    best = 10**9
    def search(row, total):
        nonlocal best
        if total >= best: return
        if row == n:
            best = total
            return
```

8-3 Python (2/2)

```
        for col in range(n):
            if not used[col]:
                used[col] = True
                search(row+1, total+cost[row][col])
                used[col] = False
    search(0, 0)
    return best
```

보수적 최악 시간 O(n·n!), 공간 O(n)

8-3 C 함수의 인자

함수 원형

int assignment_min(const int cost[][8], int n)

배열과 작업 공간

cost는 비용 표, n은 사람 수이자 작업 수이다.

8-3 C

```
static int assign_go(const int cost[][8], int n, int row,
    int used[], int total, int best) {
    if (total>=best) return best;
    if (row==n) return total;
    for (int c=0; c<n; ++c) if (!used[c]) {
        used[c]=1;
        best=assign_go(cost,n,row+1,used,total+cost[row][c],best);
        used[c]=0;
    }
    return best;
}
int assignment_min(const int cost[][8], int n) {
    int used[8]={0};
    return assign_go(cost,n,0,used,0,1000000000);
}
```

보수적 최악 시간 O(n·n!), 공간 O(n)

8-4 모든 도시를 방문하는 최소 왕복 비용

0번 도시에서 출발해 나머지 도시를 각각 한 번씩 방문한 뒤 0번으로 돌아온다. 다음에 방문할 도시의 순서는 직접 정해야 한다. dist[i][j]는 i에서 j로 가는 비용이다.

가능한 방문 순서 중 왕복 비용의 최솟값을 반환하시오.

8-4 입력과 반환값

Python: tsp_min(dist)

dist: 도시 1~9개의 정사각 비용 표. 각 비용은 0~100이며 대각선은 0

반환값

최소 왕복 비용을 정수 하나로 반환한다. 방문 순서를 반환할 필요는 없다.

선택·판정 조건

모든 도시 사이를 이동할 수 있다. 방향에 따라 비용은 달라도 된다.
중간에 같은 도시를 다시 방문하지 않는다. 마지막에 출발점 0으로 돌아오는 방문만 예외이다.
도시가 하나이면 이동 비용은 0이다.

8-4 두 경로 중 작은 비용 · 도시 두 개

두 경로 중 작은 비용

dist=[[0,2,8],[1,0,3],[4,6,0]]
반환값: 9
0,1,2,0은 9이고 0,2,1,0은 15이다. 이 중 9를 반환한다.

도시 두 개

dist=[[0,5],[8,0]]
반환값: 13
가능한 왕복은 0,1,0 하나이며 비용은 13이다.

8-4 모든 이동이 무료 · 도시 하나

모든 이동이 무료

세 도시 사이의 모든 비용이 0
반환값: 0
어느 순서로 방문해도 비용은 0이다.

도시 하나

dist=[[0]]
반환값: 0
추가로 방문할 도시가 없다.

세 도시에서 가능한 순회는 두 가지

출발	도착0	도착1	도착2
도시0	0	2	8
도시1	1	0	3
도시2	4	6	0

출발점 0은 고정한다. 나머지 도시 1,2의 순서를 바꾸어 본다.

8-4 예제의 계산 과정

방문과 귀환	더하는 비용	합
0→1→2→0	2+3+4	9
0→2→1→0	8+6+1	15

두 순회 중 작은 9를 반환한다. 마지막 도시에서 0으로 가는 비용도 포함한다.

8-4 방문하지 않은 도시를 하나씩 골라 본다

현재 도시와 방문 표시를 저장하고, 다음 도시 후보를 하나 고른다. 모든 도시를 방문했을 때 마지막 귀환 비용까지 더해 이미 찾은 최솟값과 비교한다. 아직 남은 도시가 있어도 현재 비용이 최솟값 이상이면 더 진행하지 않는다. 각 후보를 본 뒤 방문 표시를 되돌려야 다른 순서도 살펴볼 수 있다.

8-4 코드에서 사용할 값

변수	뜻
last	지금 서 있는 도시
count	0번을 포함해 지금까지 방문한 도시 수
total	현재까지 이동한 비용
used	방문한 도시 표시. 되돌아올 때 해제

8-1은 경로 하나의 비용, 8-4는 가능한 경로 중 최소 비용을 구합니다.

8-4 Python (1/2)

```
def tsp_min(dist):
    n = len(dist)
    used = [False]*n
    used[0] = True
    best = 10**9
    def search(last, count, total):
        nonlocal best
        if total >= best: return
        if count == n:
            best = min(best, total+dist[last][0])
            return
```

8-4 Python (2/2)

```
        for nxt in range(1, n):
            if not used[nxt]:
                used[nxt] = True
                search(nxt, count+1, total+dist[last][nxt])
                used[nxt] = False
    search(0, 1, 0)
    return best
```

보수적 최악 시간 $O(n \cdot (n-1)!)$, 공간 $O(n)$

8-4 C 함수의 인자

함수 원형

int tsp_min(const int dist[][9], int n)

배열과 작업 공간

dist는 비용 표, n은 도시 수이다.

8-4 C (1/2)

```
static int tour_go(const int d[][9], int n, int last,
    int count, int used[], int total, int best) {
    if (total>=best) return best;
    if (count==n) {
        int v=total+d[last][0];
        return v<best?v:best;
    }
    for (int x=1; x<n; ++x) if (!used[x]) {
        used[x]=1;
        best=tour_go(d,n,x,count+1,used,total+d[last][x],best);
        used[x]=0;
    }
    return best;
}
```

8-4 C (2/2)

```
int tsp_min(const int dist[][9], int n) {
    int used[9]={0};
    used[0]=1;
    return tour_go(dist,n,0,1,used,0,1000000000);
}
```

보수적 최악 시간 O(n·(n−1)!), 공간 O(n)